Handwritten Digit Generation Using GAN

1. Objective

The goal of this assignment is to build a model that can create images of handwritten digits similar to the MNIST dataset.

A **Generative Adversarial Network (GAN)**, is a system made of two models that compete with each other.One model is called the **generator**. It creates fake images. The other model is called the **discriminator**. It tries to tell if an image is real or fake.

At the start, the generator produces very bad images. The discriminator easily detects them. But over time, the generator improves by learning from its mistakes. At the same time, the discriminator also improves.

This back and forth process continues until the generator produces images that look close to real handwritten digits.

2. Architecture of Generator and Discriminator

Generator

The generator takes random noise as input and converts it into an image.

In this assignment, the generator uses:

- A dense layer to expand the input
- Batch normalization to stabilize learning
- Leaky ReLU activation for better gradient flow
- Reshape layer to convert data into image format
- Transposed convolution layers to increase image size step by step

The final output is a 28 by 28 grayscale image, similar to MNIST digits.

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12544)	1,254,400
batch_normalization (BatchNormalization)	(None, 12544)	50,176
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 128)	819,200
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	1,600

Discriminator

The discriminator takes an image as input and decides if it is real or fake.

It uses:

- Convolution layers to extract features
- Leaky ReLU activation
- Dropout to prevent overfitting
- Flatten layer to convert data into a single vector
- A dense layer with sigmoid activation to output a probability

If the output is close to one, the image is considered real. If it is close to zero, it is considered fake.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6,273

3. Loss and Training Behavior

Two loss functions are used. The **discriminator loss** measures how well it can tell real and fake images apart. The **generator loss** measures how well it can fool the discriminator.

During training:

- The discriminator learns to correctly classify images
- The generator learns to produce better images

At the beginning, discriminator loss is low and generator loss is high. This means the discriminator is winning. Over time, generator loss decreases as it improves.

If training is balanced, both losses reach a stable point.

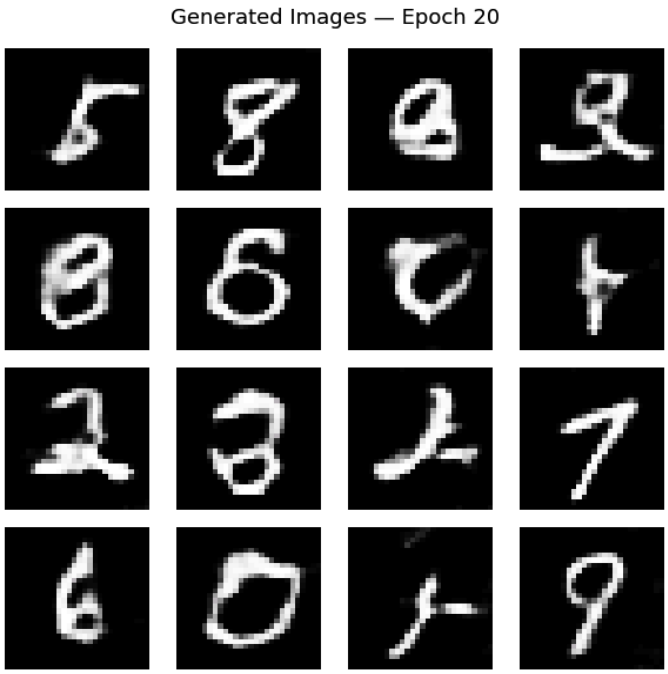
4. Generated Outputs

At early stages, generated images look like random noise.

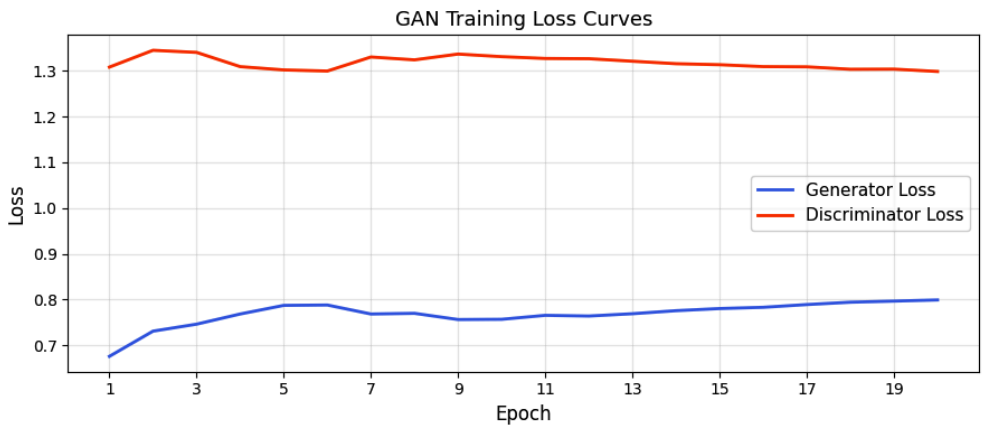
After some training:

- Shapes of digits start to appear
- Images become clearer
- Some digits become recognizable

By the end, many generated images look similar to real handwritten digits, although not perfect.



Loss Graph



5. Observations and Challenges

Training a GAN is not straightforward.

Some key observations:

- The generator and discriminator must stay balanced
- If the discriminator becomes too strong, the generator cannot learn
- If the generator becomes too strong, the discriminator fails

Challenges faced:

- Training instability
- Loss values can fluctuate instead of smoothly decreasing
- Generated images may sometimes repeat patterns
- Requires careful tuning of learning rate and batch size

Another issue is that results improve slowly and require many training steps.

Conclusion

This assignment shows how GANs can learn to generate realistic images from random noise. Even though the model is simple, it demonstrates the core idea of adversarial learning.

GANs are powerful but difficult to train. Proper balance between generator and discriminator is the key to good results.